# most-asked-questions

# Most-Asked CAT and Exam Questions (Days 1 to 4)

Forty questions that come up again and again in CATs and exams on foundations, Hadoop, Hive, Pig and Spark, with short answers you can write in an exam.

---

## 1. Foundations

### Q1. Define Big Data using the 5 V's. Give an example of each.

Big Data is data whose size, speed or variety is too much for one machine or a traditional database to store and analyse in acceptable time and cost.

| V | Meaning | Example (mobile money operator) |
|---|---|---|
| Volume | Amount of data | Billions of transactions per year |
| Velocity | Speed it arrives and must be handled | Thousands of payments per second, fraud checks in real time |
| Variety | Different formats | Transaction tables, SMS text, app logs (JSON), call audio |
| Veracity | Quality and trust | Duplicate records, missing agent IDs, wrong timestamps |
| Value | Benefit after analysis | Detecting fraud, predicting churn |

### Q2. Name the four types of analytics and give one example of each.

| Type | Question it answers | Example |
|---|---|---|
| Descriptive | What happened? | Monthly sales report per region |
| Diagnostic | Why did it happen? | Drill-down shows sales fell because one store closed |
| Predictive | What will happen? | Forecast next month's demand |
| Prescriptive | What should we do? | Decide how much stock to send to each store |

Each level is harder to build and gives more business value than the one before.

### Q3. Compare vertical and horizontal scaling. Which one do Big Data systems use and why?

- **Vertical (scale up):** buy a bigger machine (more CPU, RAM, disk). Simple, no code change, but there is a hardware ceiling and cost rises fast.
- **Horizontal (scale out):** add more ordinary machines to a cluster. No hard ceiling and cheaper per unit, but you must deal with partitioning, network failures and consistency.

Big Data platforms (HDFS, Spark, Cassandra) scale out, because data grows past what any single machine can hold and commodity servers are cheaper.

### Q4. State the CAP theorem. Where do HBase, Cassandra and MongoDB sit, and why can a distributed system not simply choose CA?

CAP: during a network partition a distributed data store cannot give both Consistency (every read sees the latest write) and Availability (every request gets a response). Partition tolerance is not optional, because networks do fail; so the real choice during a partition is C or A.

| System | Usual placement | Why |
|---|---|---|
| HBase | CP | Each region is served by one RegionServer; if it is unreachable, that data is unavailable until recovery |
| MongoDB | CP (default, majority write concern) | Writes go to the primary; during an election writes pause |
| Cassandra | AP | Any replica accepts writes; replicas reconcile later. Consistency is tunable per query (QUORUM, ALL) |

"CA" only exists for a single-node database, where there is no network partition to tolerate.

### Q5. Compare ACID and BASE. Give a use case for each.

| | ACID | BASE |
|---|---|---|
| Stands for | Atomicity, Consistency, Isolation, Durability | Basically Available, Soft state, Eventual consistency |
| Priority | Correctness of every transaction | Availability and scale |
| Typical systems | PostgreSQL, MySQL, Oracle | Cassandra, DynamoDB, many NoSQL stores |
| Use case | Bank transfer: money must never appear or vanish | Social media likes counter: being a few seconds stale is fine |

### Q6. Compare the Lambda and Kappa architectures.

- **Lambda:** two paths. A batch layer recomputes accurate results from all historical data; a speed layer processes recent data in real time; a serving layer merges both. Accurate and fresh, but you write and maintain the same logic twice.
- **Kappa:** one streaming path for everything. To reprocess history you replay the stored stream (for example from Kafka). One code base, but it needs a durable, replayable log and mature streaming tools.

Choose Lambda when batch accuracy is critical and the team already has batch jobs. Choose Kappa when most logic is event-driven and the log can hold enough history.

---

## 2. HDFS and YARN

### Q7. Distinguish the NameNode, the Secondary NameNode and the Standby NameNode.

| | Role | Is it a hot backup? |
|---|---|---|
| NameNode (active) | Holds the namespace and block map in memory; answers all metadata requests | It is the master |
| Secondary NameNode | Periodically merges the edit log into the fsimage (checkpointing) so restarts are fast | **No.** It cannot take over |
| Standby NameNode (HA) | Keeps an up-to-date copy of metadata via JournalNodes; ZooKeeper failover promotes it if the active fails | Yes |

The Secondary NameNode is a common trick question: despite the name, it is a checkpoint helper, not a failover node.

### Q8. Why is the HDFS block size large (128 MB) compared with a normal file system (4 KB)?

- **Less metadata:** the NameNode keeps every block in RAM. Bigger blocks mean fewer blocks per file and less memory.
- **Disk efficiency:** large sequential reads spend most of their time transferring data, not seeking.
- **Sensible task size:** one map task usually processes one block, so 128 MB gives tasks that run long enough to be worth scheduling.

Too large a block size reduces parallelism (fewer tasks); too small overloads the NameNode.

### Q9. A 1 GB file is stored in HDFS with 128 MB blocks and replication factor 3. How many blocks, block replicas and raw storage? What changes for a 300 MB file?

- 1 GB = 1024 MB. 1024 / 128 = **8 blocks**.
- Each block has 3 copies: 8 × 3 = **24 block replicas**.
- Raw storage: about 1 GB × 3 = **3 GB**.

For a 300 MB file: ceil(300 / 128) = 3 blocks (128, 128, 44 MB), 9 replicas, about 900 MB raw. The last block only uses 44 MB on disk; HDFS does not pad it to 128 MB.

### Q10. What is the small files problem and how do you fix it?

Every file, directory and block is an object in NameNode memory (roughly 150 bytes each). Millions of tiny files fill the NameNode heap even when the total data is small. They also create one map task (or Spark task) per file, so jobs spend more time starting tasks than processing data.
Fixes:

- merge small files before or after loading (a compaction job);
- write with fewer output partitions (`coalesce` in Spark);
- store data in container formats such as ORC, Parquet, Avro or SequenceFiles;
- use Hadoop Archives (HAR) for cold data;
- avoid over-partitioning Hive tables.

### Q11. What happens when a DataNode dies?

1. The DataNode stops sending heartbeats (every 3 seconds by default).
2. After about 10 minutes without heartbeats, the NameNode marks it dead.
3. The NameNode finds every block that now has fewer than 3 live replicas.
4. It tells other DataNodes to copy those blocks from surviving replicas until the replication factor is restored.

Clients that were reading from the dead node simply switch to another replica. No data is lost unless all replicas of a block fail at the same time.

### Q12. When you run Spark on YARN, what is the difference between client mode and cluster mode?

| | **Client mode** | **Cluster mode** |
|---|---|---|
| Where the driver runs | On the machine where you ran `spark-submit` | Inside a YARN container (in the ApplicationMaster) |
| Output of `print` / `show` | Your terminal | YARN container logs |
| If your laptop disconnects | Job dies | Job keeps running |
| Use for | Interactive work, notebooks, debugging | Scheduled production jobs |

```
spark-submit --master yarn --deploy-mode cluster job.py
```

---

## 3. MapReduce

### Q13. Distinguish a combiner from a partitioner.

| | Combiner | Partitioner |
|---|---|---|
| Purpose | Shrink map output before the shuffle | Decide which reducer receives each key |
| Runs | On the mapper side, on one mapper's output | On the mapper side, for every output record |
| Default | None (optional) | Hash partitioner: `hash(key) mod R` |
| Guarantee | May run zero, one or many times | Always runs when there are reducers |
| Affects | Network traffic | Load balance between reducers and which keys meet |

A combiner is only safe for associative and commutative operations (sum, count, max). A custom partitioner is used to send related keys together or to spread a hot key.

### Q14. What is speculative execution?

When one task runs much slower than others in the same phase (a straggler), the framework launches a duplicate copy of it on another node. Whichever copy finishes first is used and the other is killed.

It helps when the slowness comes from a bad node (failing disk, busy machine). It does not help with data skew, because the duplicate gets the same oversized input. It uses extra resources, so it is often turned off on busy clusters. In Hadoop it is on by default; in Spark it is off by default (`spark.speculation=false`).

### Q15. Trace Word Count on the two input lines "big data is big" and "data is fast".

Map output:

- line 1: (big,1) (data,1) (is,1) (big,1)
- line 2: (data,1) (is,1) (fast,1)

After a combiner on mapper 1: (big,2) (data,1) (is,1).
Shuffle and sort (grouped by key):

- big → [2]
- data → [1, 1]
- fast → [1]
- is → [1, 1]

Reduce (sum): (big,2) (data,2) (fast,1) (is,2).

### Q16. Why is it wrong to use "average" as a combiner, and how do you fix it?

The average of averages is not the overall average when groups have different sizes. Example: mapper A sees 10 and 20 (avg 15), mapper B sees 60 (avg 60). Average of averages = 37.5, but the true average of 10, 20, 60 is 30.

Fix: emit (sum, count) pairs. The combiner adds sums and counts; the reducer adds them again and divides once at the end: (30, 2) + (60, 1) = (90, 3), so 90 / 3 = 30.

### Q17. Why is MapReduce slow for iterative algorithms such as k-means or PageRank?

Each iteration is a separate MapReduce job. Every job reads its input from HDFS, writes map output to local disk, shuffles, and writes the final output back to HDFS with 3 replicas. The next iteration reads it all again, and each job also pays start-up and scheduling cost. With 50 iterations, most of the time goes to disk I/O and job overhead, not computation. Spark fixes this by caching the dataset in memory between iterations.

---

## 4. Hive and Pig

### Q18. A team lands JSON event logs in HDFS every hour. How can Hive query them without first converting them into tables?

Hive uses **schema-on-read**. You create an external table that describes the columns and points at the folder, with a JSON SerDe to parse each line. The files are not changed or validated when they land; the schema is applied only when a query reads them.

```
CREATE EXTERNAL TABLE events (user_id STRING, action STRING, ts TIMESTAMP)
ROW FORMAT SERDE 'org.apache.hive.hcatalog.data.JsonSerDe'
LOCATION '/data/raw/events';
```

Trade-off: loading is instant and flexible, but bad records only show up as NULLs or errors at query time.

### Q19. Compare managed and external tables. What happens on DROP TABLE?

| | Managed (internal) | External |
|---|---|---|
| Who owns the files | Hive | Someone else (Hive only reads them) |
| Location | Hive warehouse directory | Any path given with `LOCATION` |
| `DROP TABLE` | Deletes metadata **and** data files | Deletes metadata only; files stay |
| Use for | Tables Hive creates and controls | Shared raw data used by other tools (Spark, Pig) |

### Q20. Compare ORC, Parquet and Avro.

| | ORC | Parquet | Avro |
|---|---|---|---|
| Layout | Columnar | Columnar | Row-based |
| Best for | Hive analytics, Hive ACID tables | Analytics across many engines (Spark, Trino, Impala) | Whole-record pipelines, Kafka, ETL |
| Reads only needed columns | Yes | Yes | No |
| Predicate pushdown with min/max stats | Yes | Yes | No |
| Schema evolution | Good | Good | Very good |

Rule of thumb: Avro for moving records, Parquet or ORC for querying them.

### Q21. Write a Hive table partitioned by date, load it with dynamic partitioning, and write a query that uses partition pruning.

```
CREATE TABLE sales (sale_id BIGINT, region STRING, amount DOUBLE)
PARTITIONED BY (dt STRING)
STORED AS ORC;

SET hive.exec.dynamic.partition.mode=nonstrict;
INSERT OVERWRITE TABLE sales PARTITION (dt)
SELECT sale_id, region, amount, dt FROM sales_raw;   -- partition column last

SELECT region, SUM(amount)
FROM sales
WHERE dt = '2024-03-15'          -- reads one folder only
GROUP BY region;
```

Pruning works because the filter is directly on the partition column. Wrapping it in a function (for example `WHERE year(dt) = 2024` on a string column) can stop pruning. Avoid partitioning on high-cardinality columns such as `sale_id`: it creates millions of tiny folders.

### Q22. Write Word Count in Pig Latin and explain each line.

```
lines  = LOAD 'input' AS (line:chararray);                    -- one tuple per line
words  = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) AS word; -- split into words, one per
row
groups = GROUP words BY word;                                  -- bag of rows per word
counts = FOREACH groups GENERATE group AS word, COUNT(words) AS n;
STORE counts INTO 'output';                                    -- triggers execution
```

`TOKENIZE` returns a bag of words and `FLATTEN` turns that bag into separate rows. Nothing runs until `STORE` (or `DUMP`), similar to Spark actions.

---

## 5. Spark core

### Q23. What is a DAG in Spark and how does Spark turn it into stages and tasks?

A DAG (directed acyclic graph) is the chain of transformations from the source data to the action, with no loops. When an action runs, the DAG scheduler walks the graph backwards from the result and cuts it at every wide dependency (shuffle). Each piece between cuts is a stage. Narrow steps inside a stage are pipelined, so data flows through `map` then `filter` in one pass. Each stage runs one task per partition.

### Q24. Compare cache(), persist() and checkpoint().

| | `cache()` | `persist(level)` | `checkpoint()` |
|---|---|---|---|
| Stores | Default level (RDD: memory only; DataFrame: memory and disk) | Level you choose (`MEMORY_ONLY`, `MEMORY_AND_DISK`, `DISK_ONLY`, `_2` replicated) | Reliable storage such as HDFS |
| Keeps lineage | Yes | Yes | **No**, lineage is cut |
| Survives executor loss | Recomputed from lineage | Recomputed (unless replicated level) | Yes, read back from storage |
| Use for | Data reused a few times | Fine control over memory vs disk | Long lineage in iterative jobs |

```python
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)
sc.setCheckpointDir("hdfs:///tmp/ckpt")
rdd.checkpoint()     # happens at the next action
```

### Q25. Compare repartition() and coalesce().

- `repartition(n)` does a full shuffle and gives evenly sized partitions. It can increase or decrease the count, and can partition by a column (`repartition("region")`).
- `coalesce(n)` only decreases the count by merging existing partitions on the same executor, with no full shuffle. It is cheap but partitions can end up uneven.

Use `repartition` to raise parallelism or prepare for a join on a key. Use `coalesce` before writing output to avoid many small files.

### Q26. What are broadcast variables and accumulators?

- **Broadcast variable:** a read-only value (for example a lookup dictionary) sent once to each executor instead of once with every task.
- **Accumulator:** a write-only counter that tasks add to and only the driver reads, for example to count bad records.

```
rates = sc.broadcast({"RWF": 1.0, "USD": 1300.0})
bad = sc.accumulator(0)

def convert(row):
    if row[1] not in rates.value:
        bad.add(1)
        return None
    return row[0] * rates.value[row[1]]

total = rdd.map(convert).filter(lambda x: x is not None).sum()
print(bad.value)
```

Caution: updates made inside transformations can be counted twice if a task is retried. Only updates inside actions (`foreach`) are guaranteed to be applied once.

### Q27. Count the jobs and stages (exercise 1).

```
rdd = sc.textFile("data.txt")
counts = rdd.flatMap(lambda l: l.split()).map(lambda w: (w, 1)).reduceByKey(lambda a, b: a +
        b)
counts.count()
counts.collect()
```

- Jobs: **2** (two actions: `count` and `collect`).
- Stages per job: 2 (`reduceByKey` is the only shuffle). Stage 1: `textFile`, `flatMap`, `map`. Stage 2: the reduce side.
- In the second job the first stage shows as **skipped** in the Spark UI, because its shuffle output from the first job is reused. So 4 stages are planned, 3 actually run.

### Q28. Count the jobs, stages and tasks (exercise 2).

```
a = sc.parallelize(range(100), 4).map(lambda x: (x % 10, x))
b = a.reduceByKey(lambda x, y: x + y)
c = b.map(lambda kv: (kv[1] % 3, kv[0])).groupByKey()
c.collect()
```

- Jobs: **1** (one action).
- Stages: **3** (two shuffles: `reduceByKey` and `groupByKey`).
- Tasks: 4 per stage, so **12**. Shuffle outputs keep the parent's 4 partitions when `spark.default.parallelism` is not set and no partition count is passed.

### Q29. Count the jobs and stages (exercise 3, with cache).

```
from operator import add
errors = sc.textFile("logs").filter(lambda l: "ERROR" in l).cache()
errors.count()                                                  # action 1
errors.filter(lambda l: "disk" in l).count()                    # action 2
errors.map(lambda l: (l.split()[0], 1)).reduceByKey(add).collect()  # action 3
```

- Jobs: **3**.
- Job 1: 1 stage. Reads the file, filters, and fills the cache.
- Job 2: 1 stage. Reads from the cache, not the file.
- Job 3: 2 stages (shuffle at `reduceByKey`). The first stage reads from the cache.
- Total: **4 stages**. Without `cache()` the log file would be read 3 times.

For DataFrames the count is less exact: `show()`, schema inference, broadcast joins and AQE can start extra small jobs. Use the Spark UI to check.

---

## 6. Spark SQL and optimization

### Q30. Name the phases a Spark SQL query goes through before it runs.

1. **Parse:** SQL text or DataFrame calls become an unresolved logical plan.
2. **Analyze:** table and column names are checked against the catalog and schema.
3. **Optimize (logical):** Catalyst rules rewrite the plan (push filters down, prune columns, fold constants).
4. **Physical planning:** choose concrete operators, for example broadcast join or sort-merge join, using size estimates.
5. **Code generation:** Tungsten compiles stages into Java bytecode, then tasks run on executors.

SQL and the DataFrame API go through exactly the same steps, so they perform the same.

### Q31. What is Adaptive Query Execution (AQE)?

AQE re-optimizes a query while it runs, using real statistics from finished shuffle stages instead of guesses. It is on by default from Spark 3.2. It does three main things:

- merges many small shuffle partitions into fewer, larger ones;
- switches a sort-merge join to a broadcast join when one side turns out to be small;
- splits skewed partitions in joins into smaller pieces.

In `explain()` you see `AdaptiveSparkPlan`.

### Q32. Why are Python UDFs slow, and what should you use instead?

A plain Python UDF sends each row from the JVM to a Python worker process and back. That serialization is costly, and the function is a black box to Catalyst, so no pushdown or code generation happens through it. Better options, in order:

1. built-in functions (`pyspark.sql.functions`: `when`, `regexp_extract`, `date_add`, etc.);
2. `pandas_udf` (vectorized, uses Apache Arrow, processes batches instead of single rows);
3. a Scala or Java UDF registered in Spark, if the logic is complex.

### Q33. What is the difference between predicate pushdown and partition pruning?

- **Partition pruning:** skips entire folders because the filter is on the partition column (`dt = '2024-03-15'` reads only `dt=2024-03-15/`). Shown as `PartitionFilters` in `explain()`.
- **Predicate pushdown:** sends a filter on a normal column into the file reader. Parquet and ORC use min/max statistics per row group to skip blocks that cannot match. Shown as `PushedFilters`.

Both reduce data read. Pruning works at folder level using metadata; pushdown works inside files.

### Q34. What does spark.sql.shuffle.partitions control, and how should you set it?

It sets how many partitions (and so tasks) a DataFrame shuffle creates for joins and aggregations. The default is 200.

- Too high for small data: hundreds of tiny tasks and tiny output files; overhead dominates.
- Too low for big data: huge partitions that spill to disk or run out of memory.

A common target is 100 to 200 MB of shuffle data per partition, and at least 2 to 3 times the total number of cores. With AQE on, set it on the high side and let AQE merge small partitions.

---

## 7. MLlib and GraphX

### Q35. Explain Transformer, Estimator and Pipeline with an example.

- **Transformer:** has `transform(df)` and returns a new DataFrame, for example `VectorAssembler` or a fitted model.
- **Estimator:** has `fit(df)` and returns a Transformer, for example `StringIndexer` or `LogisticRegression`.
- **Pipeline:** a list of stages. `fit` runs them in order and returns a `PipelineModel` that applies the same steps to new data.

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.classification import LogisticRegression

idx = StringIndexer(inputCol="category", outputCol="category_idx")
vec = VectorAssembler(inputCols=["category_idx", "age_days"], outputCol="features")
lr = LogisticRegression(featuresCol="features", labelCol="label")

model = Pipeline(stages=[idx, vec, lr]).fit(train)
predictions = model.transform(test)
```

Pipelines prevent a common bug: training and prediction data being prepared differently.

### Q36. A fraud model shows 99% accuracy but catches almost no fraud. Why, and which metrics should you report?

Because the classes are imbalanced. If only 1% of transactions are fraud, a model that always says "not fraud" is 99% accurate and useless.

Report instead:

- **Recall:** of all real fraud, how much did we catch?
- Precision: of all flagged cases, how many were real fraud?
- F1 score (balance of the two) and area under the precision-recall curve.
- A confusion matrix.

Fixes include class weights, resampling, and choosing the decision threshold based on cost.

### Q37. Explain PageRank and the Pregel (superstep) model in simple terms.

PageRank scores a vertex by how many important vertices point to it. Every vertex starts with the same score. In each round it sends its score, split evenly, along its outgoing edges; each vertex adds up what it receives, applies a damping factor (usually 0.85), and repeats until scores stop changing.

Pregel runs graph algorithms this way: in each **superstep** every vertex reads messages from the last step, updates its value, and sends messages to neighbours. The algorithm stops when no messages remain or after a maximum number of steps. Each superstep is a distributed round with a barrier, so the number of iterations drives the runtime.

---

## 8. Choosing the right tool (scenario questions)

### Q38. For each scenario, pick MapReduce, Hive, Pig or Spark and justify.

| Scenario | Pick | Reason |
|---|---|---|
| Analysts need a monthly report over 5 TB of sales in a data lake | Hive (or Spark SQL) | SQL-natural, shared catalog, batch latency is fine |
| Train a churn model with 30 passes over 2 TB | Spark (MLlib) | Iterative; caching avoids rereading disk each pass |
| Legacy ETL already written as MapReduce, runs fine nightly | Keep MapReduce | Works; rewriting costs more than it saves |
| Multi-step cleaning of nested, messy logs by an engineering team | Pig or Spark DataFrames | Step-by-step dataflow; Spark is the modern choice |
| Rank users in a 2 billion edge social graph | Spark GraphX or GraphFrames | Built-in iterative graph algorithms |
| Sub-second dashboard over 10 GB | None of these | Use DuckDB, ClickHouse, Druid or precomputed tables |

### Q39. Design a small batch analytics pipeline for a court case system (sources to dashboard), naming a tool for each layer.

1. **Sources:** the case management database (PostgreSQL), hearing logs, uploaded documents.
2. **Ingestion:** a nightly export of changed rows (Sqoop or Spark JDBC read); logs shipped as files.
3. **Storage:** raw zone in HDFS or object storage as received; curated zone as Parquet partitioned by date.
4. **Processing:** Spark jobs clean, join cases and hearings, and compute backlog per court and category.
5. **Analytics:** Spark SQL or Hive tables for analysts; MLlib to predict which cases risk long delays.
6. **Consumption:** small aggregate tables feed a dashboard (Superset, Power BI).
7. **Governance and operations:** Airflow schedules and retries the jobs; access control on sensitive case data; data quality checks; lineage recorded in a catalog.

Consistency choice: the source case database stays ACID (CP); the analytics copy can be a day old, which is acceptable for reporting.

### Q40. Your whole dataset is 2 GB and fits on a laptop. Should you use Hadoop or Spark?

Usually no. At this size, pandas, Polars or DuckDB on one machine are faster and simpler: no cluster to run, no shuffle, no job overhead. Distributed tools pay off when data does not fit on one machine, when processing on one machine takes too long, or when the data is growing fast toward that point. In an exam, state the size threshold, the cost of running a cluster, and the growth expectation as your reasons.

---

## Quick recall

- 5 V's: Volume, Velocity, Variety, Veracity, Value.
- CAP: during a partition choose C or A. HBase and MongoDB lean CP; Cassandra leans AP.
- Secondary NameNode checkpoints metadata; Standby NameNode is the real failover.
- HDFS default: 128 MB blocks, replication 3, heartbeat every 3 s.
- Blocks = ceil(file size / block size); raw storage is about file size × replication.
- Combiner reduces shuffle volume; partitioner decides the reducer (`hash(key) mod R`).
- Never average averages: carry (sum, count).
- Dropping an external Hive table keeps the files; dropping a managed table deletes them.
- Partition pruning skips folders; predicate pushdown skips blocks inside files.
- One action = one job; each shuffle adds a stage; one task per partition.

- `coalesce` to reduce files cheaply; `repartition` to rebalance with a shuffle.
- Accuracy misleads on imbalanced data; report recall, precision and F1.